Rapport — Optimisation par Colonie de Fourmis (ACO) sur un Paysage Fractal

Membres de l'équipe :

- João Victor Ferro Amorosini
- Vinicius César Cavallaro Zancheta
- Edilberto Elias Xavier Junior

Date : 12/03/2026
Machine : Intel i7-11800H — 8 cœurs physiques / 16 threads logiques (hyper-threading), 20 Mo cache L3
Compilateur : g++ / mpic++ avec `-O3 -march=native -fopenmp -std=c++17`

---

1. Objectif du Projet

Ce projet est un exercice classique de Calcul Haute Performance (HPC). L'objectif **n'est pas seulement** de créer une simulation de fourmis, mais de transformer un code séquentiel simple en un code extrêmement rapide et efficace, capable de s'exécuter sur plusieurs cœurs (OpenMP) et sur plusieurs ordinateurs simultanément (MPI).

Le travail est organisé en quatre étapes progressives :

Étape	Description
A	Implémenter le modèle, le terrain fractal et mesurer le temps par itération (référence séquentielle)
B	Vectorisation : réorganiser les données des fourmis en SoA (Structure of Arrays)
C	Parallélisme à Mémoire Partagée (OpenMP)
D	Parallélisme à Mémoire Distribuée (MPI)

2. Contexte Expérimental

2.1 Matériel et Logiciel

Élément	Valeur
CPU	Intel i7-11800H
Cœurs physiques / threads	8 / 16 (hyper-threading)
Cache L3	20 Mo (partagé)
Compilateur	g++ 11 / mpic++ (OpenMPI 4.1.6)
Flags	`-O3 -march=native -fopenmp -std=c++17`
OS	Linux (Ubuntu 22.04)

2.2 Compilation

```
cd projet/src && make clean && make all
```

Deux exécutables sont générés :

- `ant_simu.exe` — version avec SDL2 (interactive) et support OpenMP
- `ant_simu_mpi.exe` — version MPI + OpenMP (sans GUI)

2.3 Paramètres fixes des benchmarks

Paramètre	Valeur
Dimension de la carte	512 × 512 cellules
Fourmis (OO/SoA)	5000
Fourmis (MPI)	20000
Exploration (ε)	0.8
Bruit du phéromone (α)	0.7
Évaporation (β)	0.999
Graine RNG	2026
Itérations (N)	1000 (OO/SoA/OMP), 300 (MPI)
Exécutions par configuration	5 (OO/SoA/OMP), 3 (MPI)

2.4 Méthodologie de mesure

Pour garantir la reproductibilité :

- Graine fixée (`--seed 2026`) et mode sans GUI (`--no-gui`) pour éviter la variabilité liée à SDL.
- **5 exécutions** par configuration. Nous rapportons la **moyenne** et l'**écart-type** du temps par itération.
- OpenMP contrôlé avec `OMP_NUM_THREADS`. Pour des benchmarks rigoureux : `OMP_PROC_BIND=true OMP_PLACES=cores`.
- L'outil `python3 tools/bench.py` automatise la collecte et calcule la moyenne/l'écart-type.

Remarque méthodologique : lorsque la différence entre deux versions est inférieure à ~2%, les résultats avec `N=200` et seulement 3 exécutions deviennent très sensibles au bruit du système. Pour la comparaison OO vs SoA, un benchmark de **1000 itérations** et multiples exécutions avec `OMP_NUM_THREADS=1` a été utilisé.

---

3. Étape A — Le Modèle et la Simulation Séquentielle

3.1 Description du Modèle ACO

3.1.1 Le Terrain

Le terrain est une grille 2D de 512×512 cellules. Chaque cellule possède un **coût de traversée** $c(s) \in [0, 1]$ — plus il est élevé, plus elle est difficile à traverser. Il y a quatre types de cellules :

- **Fourmilière (nid) :** cellule d'origine
- **Source de nourriture :** destination
- **Indésirables (murs) :** valeur −1, les fourmis ne peuvent pas y entrer
- **Libres :** explorables

3.1.2 Les Fourmis

Chaque fourmi peut avoir deux états possibles :

- **Non-chargée :** cherche de la nourriture en suivant le phéromone V_1
- **Chargée :** retourne au nid en suivant le phéromone V_2

Règles de transition d'état :

- La fourmi arrive à la nourriture → devient **chargée**
- La fourmi arrive au nid **chargée** → incrémente le compteur de nourriture, devient **non-chargée**

3.1.3 Les Phéromones

Chaque cellule s stocke deux valeurs : $V_1(s)$ (guide vers la nourriture) et $V_2(s)$ (guide vers le nid).

Mise à jour par une fourmi en s :

$$V_1(s) \leftarrow \begin{cases} 1 & \text{si } s \text{ est la nourriture} \\ \alpha \cdot \max_{s' \in N(s)} V_1(s') + (1-\alpha) \cdot \frac{1}{4}\sum_{s' \in N(s)} V_1(s') & \text{sinon} \end{cases}$$

$$V_2(s) \leftarrow \begin{cases} 1 & \text{si } s \text{ est le nid} \\ \alpha \cdot \max_{s' \in N(s)} V_2(s') + (1-\alpha) \cdot \frac{1}{4}\sum_{s' \in N(s)} V_2(s') & \text{sinon} \end{cases}$$

Où $\alpha = 0.7$ est le paramètre de bruit et $N(s)$ sont les 4 cellules voisines (gauche, droite, haut, bas).

Évaporation à la fin de chaque itération : $$V_i(s) \leftarrow \beta \cdot V_i(s), \quad \beta = 0.999$$

3.1.4 La Règle de Déplacement

À chaque sous-étape dans le temps, la fourmi possède 1 unité de mouvement qu'elle dépense selon le coût du terrain $c(s)$. Elle répète les sous-étapes jusqu'à épuiser son mouvement. À chaque sous-étape :

- **Avec probabilité $\varepsilon = 0.8$** (exploration) : avance vers une cellule voisine **aléatoire** valide (valeur ≠ −1)
- **Avec probabilité $1-\varepsilon = 0.2$** (guidée) : avance vers le voisin ayant le plus grand $V_i(s)$ (V_1 si non-chargée, V_2 si chargée)

Le paramètre ε est le **coefficient d'exploration**. Un seuil de 10^{-3} est appliqué au coût du terrain pour éviter les boucles infinies dans les cellules au coût proche de zéro.

3.2 Le Terrain Fractal (Algorithme de Plasma)

Le terrain est généré par l'algorithme **diamond-square** (déplacement récursif) :

1. Des altitudes aléatoires sont générées aux **coins** des sous-grilles de taille $ns = 2^k$, avec une déviation maximale $d \cdot ns$.
2. **Récursivement**, pour chaque sous-grille :
   - On calcule le **point médian de chaque bord** (interpolation + bruit avec déviation $d \cdot ns/2$)
   - On calcule le **point central** (moyenne des 4 points de bord + bruit)
3. Répéter récursivement jusqu'aux sous-grilles 2×2.
4. **Normalisation :** toutes les valeurs sont remappées sur $[0, 1]$.

Code dans `src/fractal_land.cpp` :

```
// Pour chaque sous-grille de niveau ldim, nous calculons les points
médians :
cur_land(i_mid, jBeg) = 0.5*(cur_land(iBeg,jBeg) + cur_land(iEnd,jBeg)) +
mid_ind*gen(i_mid,jBeg);
cur_land(iBeg, j_mid) = 0.5*(cur_land(iBeg,jBeg) + cur_land(iBeg,jEnd)) +
mid_ind*gen(iBeg,j_mid);
cur_land(i_mid, j_mid) = 0.25*(/* 4 points de bord */) +
mid_ind*gen(i_mid,j_mid);
```

Le résultat : les régions claires dans la visualisation = terrain difficile (coût élevé), régions sombres = facile.

3.3 Structure du Code

Fichier	Responsabilité
`ant.hpp / ant.cpp`	Logique d'une fourmi : état, position, `advance()`
`pheronome.hpp`	Carte des phéromones, évaporation, double-buffer, `mark_pheronome()`
`fractal_land.hpp / .cpp`	Génération algorithmique du terrain fractal
`ant_simu.cpp`	Boucle principale (séquentiel + SoA + OpenMP) + instrumentation du temps
`ant_simu_mpi.cpp`	Boucle principale avec MPI + OpenMP

Boucle principale (version séquentielle) :

```
for (std::size_t it = 0; it < opt.steps; ++it) {
    // Phase 1 : chaque fourmi se déplace et dépose des phéromones
    for (auto& a : ants)
        a.advance(phen, land, pos_food, pos_nest, food_quantity);

    // Phase 2 : évaporation des phéromones  V_i(s) *= beta
    phen.do_evaporation();

    // Phase 3 : échange de buffers + conditions aux limites
```

```
    phen.update();
}
```

3.4 Instrumentation du Temps

En utilisant `std::chrono::steady_clock`, chaque phase est mesurée individuellement :

```
auto t0 = clock::now();
for (auto& a : ants) a.advance(phen, land, pos_food, pos_nest,
food_quantity);
auto t1 = clock::now();
phen.do_evaporation();
auto t2 = clock::now();
phen.update();
auto t3 = clock::now();

timings.ants_s   += duration<double>(t1 - t0).count();
timings.evap_s   += duration<double>(t2 - t1).count();
timings.update_s += duration<double>(t3 - t2).count();
```

3.5 Corrections Importantes de Robustesse

Avant de mesurer les performances, des corrections ont été nécessaires :

Problème	Cause	Correction Appliquée
Boucle infinie dans substeps	Coût du terrain = 0 possible	Seuil plancher `k_min_step_cost = 1e-3`
Underflow d'indice (bug)	`x-1` avec `size_t` quand `x=0`	Surcharge `operator()(int,int)` avec ghost cells
RNG non déterministe	`m_seed` non initialisée dans le constructeur	`ant(pos, seed) : m_seed(seed)`
Buffer incohérent	Buffer non copié après `update()`	Ajout de `sync_buffer_from_map()`

3.6 Résultats de Base (Étape A)

```
python3 tools/bench.py --steps 1000 --runs 5 --threads 1 --seed 2026
```

```
== OO OMP_NUM_THREADS=1 steps=1000 runs=5 ==
ants.advance     mean=1.020645e-03 s  std=2.609209e-05 s
pheromone.evap   mean=2.218520e-04 s  std=5.604663e-06 s
pheromone.update mean=1.831760e-04 s  std=4.419829e-06 s
```

Répartition du temps total (1.4257e-03 s/itér) :

Phase	Temps (s/itér)	% du Total
`ants.advance`	1.021e-03 ± 2.61e-05	**71.6%**
`pheromone.evap`	2.219e-04 ± 5.60e-06	15.6%
`pheromone.update`	1.832e-04 ± 4.42e-06	12.8%
Total	**1.4257e-03**	100%

Conclusion de l'Étape A : `ants.advance` domine avec ~72% du temps. Les phases d'évaporation et d'update (~28%) sont des cibles naturelles pour la parallélisation OpenMP.

---

4. Étape B — Vectorisation (Structure of Arrays)

4.1 Qu'est-ce que le SoA et Pourquoi l'utiliser ?

La représentation standard est **Array of Structures (AoS)** : les données de chaque fourmi sont conservées ensemble en mémoire. En itérant sur les fourmis et en utilisant uniquement les coordonnées, le processeur charge des octets inutiles (état, graine) dans le cache.

La **Structure of Arrays (SoA)** sépare les champs dans des tableaux indépendants :

```
// AoS — avant (Array of Structures) :
struct ant { int x, y; unsigned char loaded; std::size_t seed; };
std::vector<ant> ants;   // mémoire : [x0 y0 l0 s0 | x1 y1 l1 s1 | ...]

// SoA — après (Structure of Arrays) :
struct AntSoA {
    std::vector<int>           x;       // toutes les coord X ensemble
    std::vector<int>           y;       // toutes les coord Y ensemble
    std::vector<unsigned char> loaded;  // tous les états ensemble
    std::vector<std::size_t>   seed;    // toutes les graines ensemble
};
// mémoire : [x0 x1 x2... | y0 y1 y2... | l0 l1 l2... | s0 s1 s2...]
```

Avantage théorique : en traitant seulement `x[i]` de toutes les fourmis, le cache charge un bloc contigu de coordonnées X → meilleure utilisation de SIMD et diminution du nombre de défauts de cache (cache misses).

4.2 L'Implémentation

La version SoA se trouve dans `ant_simu.cpp` via `struct AntSoA` + fonction `advance_time_timed_soa()`, activée par :

```
./projet/src/ant_simu.exe --no-gui --vectorized --steps 200
```

La logique est **identique** à la version orientée objet : mêmes formules de phéromones, même règle de déplacement, même coût de terrain. Seule l'organisation de la mémoire change.

Lors de la révision de cette section, le kernel SoA a été ajusté pour conserver `x`, `y`, `loaded` et `seed` dans des variables locales pendant la boucle interne, limitant ainsi les lectures/écritures redondantes à chaque sous-étape. Cela rapproche mieux le coût de la version SoA de celui de la version OO.

Il est également important de distinguer deux concepts :

- **SoA** : réorganisation des données en mémoire pour améliorer la localité et préparer la vectorisation.
- **SIMD réel** : génération d'instructions vectorielles par le compilateur ou via intrinsèques.

Dans l'état actuel, le compilateur **n'a pas vectorisé automatiquement** la boucle principale des fourmis. Par conséquent, le mode `--vectorized` de ce projet doit être compris comme une **version SoA scalaire**, et non comme du SIMD explicite.

4.3 Résultats

```
export OMP_NUM_THREADS=1 OMP_PROC_BIND=true OMP_PLACES=cores
python3 tools/bench.py --steps 1000 --runs 5 --threads 1 --seed 2026
python3 tools/bench.py --soa --steps 1000 --runs 5 --threads 1 --seed 2026
```

```
== OO OMP_NUM_THREADS=1 steps=1000 runs=5 ==
ants.advance     mean=1.020645e-03 s  std=2.609209e-05 s
pheromone.evap   mean=2.218520e-04 s  std=5.604663e-06 s
pheromone.update mean=1.831760e-04 s  std=4.419829e-06 s

== SoA OMP_NUM_THREADS=1 steps=1000 runs=5 ==
ants.advance     mean=1.055970e-03 s  std=7.532138e-05 s
pheromone.evap   mean=2.349212e-04 s  std=1.572012e-05 s
pheromone.update mean=1.924872e-04 s  std=1.206314e-05 s
```

Comparaison OO vs SoA révisée (N=1000, OMP=1, seed=2026, 5 exécutions) :

Version	ants.advance	evap	update	Total (s/itér)	Vs OO
OO (standard)	1.021e-03 ± 2.61e-05	2.219e-04 ± 5.60e-06	1.832e-04 ± 4.42e-06	**1.426e-03**	1.000
SoA	1.056e-03 ± 7.53e-05	2.349e-04 ± 1.57e-05	1.925e-04 ± 1.21e-05	**1.483e-03**	1.040

4.4 Analyse et Interprétation

Lors de la dernière campagne (5 exécutions), SoA a montré une **différence de ~4.0% au total** sur 1 thread.

L'observation antérieure d'une "différence de 12% pour SoA" était influencée par deux facteurs :

1. **Méthodologie trop courte pour un effet faible.** Avec seulement 200 itérations et 3 lancements, le bruit du système était comparable à l'effet mesuré.
2. **Surcharge évitable dans la première implémentation SoA.** Le noyau lisait et réécrivait `x[idx]`, `y[idx]`, `loaded[idx]` et `seed[idx]` à l'intérieur de la boucle interne, ce qui augmentait le trafic mémoire. En passant cet état dans des variables locales, le comportement s'est aligné sur les attentes.

Le comportement observé peut varier parce que :

1. **Le goulot d'étranglement principal reste l'accès à la carte globale** des phéromones et au terrain, ce qui représente des accès irréguliers.
2. **Il y a beaucoup de branchements par fourmi** (exploration vs. guidée, chargée vs. non-chargée), ce qui complique la vectorisation automatique (SIMD).
3. Le compilateur **n'a pas généré de vectorisation SIMD réelle** pour la boucle des fourmis ; ainsi, le gain actuel du SoA vient essentiellement d'une meilleure localité et simplification d'accès.

Conclusion révisée : dans ce projet, l'organisation SoA ne produit pas d'écart majeur de performance. Le SoA prépare le code pour la vectorisation, mais sans SIMD effectif et avec des accès mémoires irréguliers sur la grille, l'effet sur 1 thread peut être mineur et sensible à l'environnement de test.

---

5. Étape C — Parallélisation avec OpenMP (Mémoire Partagée)

5.1 Identification des Boucles

Dans la version actuelle du code, les trois phases principales de l'itération utilisent OpenMP :

Phase	Parallélisable ?	Raison
`ants.advance`	✅ Oui (implémenté)	Parallélisation utilisant des buffers de marqueurs par thread, évitant les conditions de concurrence sur la carte
`pheromone.evap`	✅ Oui	`V[i][j] *= beta` — cellules complètement indépendantes
`pheromone.update`	✅ Oui	Copie de buffer — indépendance totale par cellule

5.2 Le Défi avec `ants.advance`

```
// PROBLÈME : race condition lorsque deux fourmis sont sur la même cellule
for (auto& a : ants)
    a.advance(phen, ...);  // appelle phen.mark_pheronome(pos) — écrit dans
phen[pos]
```

Plutôt que des mutex/atomics par cellule (très coûteux), une solution en deux temps a été adoptée :

1. Chaque thread traite son lot de fourmis et cumule les dépôts de phéromones dans un buffer local.
2. Après la boucle `omp for`, le programme applique les marques sur la carte de phéromones globale.

Ainsi, le déplacement des fourmis est parallélisé sans contention directe.

5.3 Pragmas Ajoutés

Déplacement des fourmis (OO et SoA) — `ant_simu.cpp`:

```
#pragma omp parallel
{
    const int tid = omp_get_thread_num();
    auto& local_marks = marks_per_thread[tid];
    std::size_t local_food = 0;

    #pragma omp for schedule(static)
    for (std::size_t i = 0; i < ants.size(); ++i)
        ants[i].advance(phen, land, pos_food, pos_nest, local_food,
&local_marks);
}
```

Dans la version SoA, le même principe est appliqué avec un `reduction(+ : food_delta)` pour le compteur de nourriture.

Évaporation — `do_evaporation()` dans `pheronome.hpp`:

```
void do_evaporation() {
    #pragma omp parallel for collapse(2) schedule(static)
    for (std::size_t i = 1; i <= m_dim; ++i)
        for (std::size_t j = 1; j <= m_dim; ++j) {
            m_buffer_pheronome[i * m_stride + j][0] *= m_beta;  // V1 *=
beta
            m_buffer_pheronome[i * m_stride + j][1] *= m_beta;  // V2 *=
beta
        }
}
```

Le `collapse(2)` fusionne les deux boucles en $512^2 = 262144$ itérations indépendantes — répartition optimale entre les threads.

Synchronisation du buffer — `sync_buffer_from_map()` dans `pheronome.hpp`:

```
void sync_buffer_from_map() {
    #pragma omp parallel for schedule(static)
    for (size_t k = 0; k < m_map_of_pheronome.size(); ++k)
        m_buffer_pheronome[k] = m_map_of_pheronome[k];
}
```

5.4 Résultats OpenMP

```
export OMP_PROC_BIND=true OMP_PLACES=cores OMP_DYNAMIC=false
python3 tools/bench.py --steps 1000 --runs 5 --threads 1 2 4 8 --seed 2026
python3 tools/bench.py --soa --steps 1000 --runs 5 --threads 1 2 4 8 --seed
2026
```

Tableau OO (N=1000, 5000 fourmis, 5 exéc., seed=2026) :

Threads p	ants.advance (s/itér)	evap (s/itér)	update (s/itér)	Total (s/itér)	$S_p = T_1/T_p$	$E_p = S_p/p$
1	1.021e-03 ± 2.61e-05	2.219e-04 ± 5.60e-06	1.832e-04 ± 4.42e-06	**1.426e-03**	1.000	1.000
2	6.376e-04 ± 1.98e-05	1.234e-04 ± 5.52e-06	1.064e-04 ± 3.85e-06	**8.673e-04**	1.644	0.822
4	6.652e-04 ± 3.86e-05	9.388e-05 ± 7.27e-06	8.635e-05 ± 8.58e-06	**8.454e-04**	1.687	0.422
8	6.793e-04 ± 5.78e-05	7.923e-05 ± 1.74e-05	7.570e-05 ± 1.79e-05	**8.342e-04**	1.709	0.214

Tableau SoA (N=1000, 5000 fourmis, 5 exéc., seed=2026) :

Threads p	ants.advance (s/itér)	evap (s/itér)	update (s/itér)	Total (s/itér)	$S_p = T_1/T_p$	$E_p = S_p/p$
1	1.056e-03 ± 7.53e-05	2.349e-04 ± 1.57e-05	1.925e-04 ± 1.21e-05	**1.483e-03**	1.000	1.000
2	6.357e-04 ± 2.74e-05	1.261e-04 ± 8.22e-06	1.090e-04 ± 7.10e-06	**8.708e-04**	1.703	0.852
4	6.936e-04 ± 2.77e-05	9.507e-05 ± 6.57e-06	8.471e-05 ± 6.52e-06	**8.734e-04**	1.698	0.425
8	6.164e-04 ± 1.22e-05	6.960e-05 ± 2.61e-06	6.523e-05 ± 1.46e-06	**7.513e-04**	1.974	0.247

5.5 Analyse des Résultats

Avec la parallélisation complète (incluant `ants.advance`), le comportement a nettement changé :

1. `ants.advance` réduit désormais de façon significative entre 1 et 2/4/8 threads (OO et SoA).
2. Le speedup total avoisine les 1.7x (OO) et 2.0x (SoA) sur 8 threads.
3. Le gain s'attaque dorénavant au véritable goulot d'étranglement de l'application.

Les phases parallélisées passent à l'échelle :

Phase	1 thread	8 threads	Réduction

Phase	1 thread	8 threads	Réduction
OO ants.advance	1.021e-03	6.793e-04	−33.4%
SoA ants.advance	1.056e-03	6.164e-04	−41.6%
OO evap	2.219e-04	7.923e-05	−64.3%
OO update	1.832e-04	7.570e-05	−58.7%

Analyse avec la loi d'Amdahl : Avant, `ants.advance` était purement séquentielle et limitait drastiquement le speedup. À présent, cette étape étant parallèle, le système s'adapte mieux aux architectures multi-cœurs.

---

6. Étape D — Parallélisation avec MPI (Mémoire Distribuée)

6.1 Approche 1 : Environnement Répliqué + Partitionnement des Fourmis

Idée

Chaque processus MPI maintient la **carte complète** (terrain + phéromones) dans sa mémoire locale et gère un sous-ensemble de M/P fourmis :

```
Processus 0 : fourmis [0, M/P)    + carte complète
Processus 1 : fourmis [M/P, 2M/P) + carte complète
...
Processus P-1 : fourmis [(P-1)M/P, M) + carte complète
```

À la fin de chaque itération, les cartes de phéromones sont synchronisées par l'opération `MPI_Allreduce(MPI_MAX)` : pour chaque cellule, on retient la valeur maximale parmi tous les processus. Prendre le `MAX` (plutôt que la somme/moyenne) est justifié, car les fourmis peuvent agir dans n'importe quel ordre.

Code de Communication

```
// Après avoir avancé les fourmis locales et mis à jour le phéromone :
MPI_Allreduce(
    phen.raw_map_doubles(),        // buffer local : (dim+2)^2 * 2 doubles
    reduced.data(),                // buffer du résultat global
    phen.raw_map_doubles_count(),  // 514*514*2 = 529,508 doubles ≈ 4.2 Mo
    MPI_DOUBLE, MPI_MAX, MPI_COMM_WORLD
);
phen.set_map_from_raw_doubles(reduced.data());
phen.sync_buffer_from_map();
```

Partitionnement sans communication de fourmis

```
void compute_local_range(size_t n_total, int rank, int n_ranks,
                         size_t& begin, size_t& end) {
    const size_t base = n_total / n_ranks;
    const size_t rem  = n_total % n_ranks;
    begin = rank * base + min(rank, rem);   // distribution round-robin du
reste
    end   = begin + base + (rank < rem ? 1 : 0);
}
```

Chaque processus génère ses fourmis de manière déterministe grâce à la graine globale. **Aucune donnée liée aux fourmis n'est communiquée**.

Utilisation

```
mpirun -np 4 ./projet/src/ant_simu_mpi.exe --steps 300 --ants 20000 --seed
2026
```

6.2 Bilan de l'Implémentation et des Benchmarks

Élément	Statut	Détail
Cible compillée	✅ OK	`make all` sans erreurs
Partitionnement	✅ OK	Distribution uniforme
`MPI_Allreduce(MAX)`	✅ OK	Ajouté et validé
Test réduit (20 steps)	✅ OK	Pas de blocage, résultat correct
Mesure de performance	✅ OK	Résumés ci-dessous

6.3 Résultats MPI

Commande exécutée (1 thread OpenMP afin d'isoler l'effet MPI) :

```
export OMP_NUM_THREADS=1
python3 tools/bench.py --mpi --steps 300 --runs 5 --ranks 1 2 4 8 --mpi-
ants 20000
```

Temps de Phase (s/itér, M=20.000, N=300) :

Rangs P	ants.advance	evap	update	MPI_Allreduce	Total (s/itér)	Speedup S_P
1	3.686e-03	2.743e-04	2.404e-06	5.992e-04	**4.562e-03**	1.000

Rangs P	ants.advance	evap	update	MPI_Allreduce	Total (s/itér)	Speedup S_P
2	2.756e-03	4.080e-04	3.432e-06	1.949e-03	**5.116e-03**	0.892
4	2.177e-03	6.511e-04	8.065e-06	5.700e-03	**8.536e-03**	0.534
8	2.256e-03	1.291e-03	1.843e-05	1.155e-02	**1.511e-02**	0.301

6.4 Discussion du Speedup MPI

Avec P processus et $M = 20000$ fourmis :

- Le trafic du processus d'Allreduce augmente avec le flux de processus.
- Le goulot s'impose drastiquement, rendant l'opération de distribution inefficace comparativement à la synchronisation volumineuse de 4.2 Mo.

Limitation : $O(n^2)$ communication, ne grandit pas du tout à l'échelle.

---

7. Stratégies de Décomposition par Domaine

Pour surmonter le goulot d'étranglement de l'approche `MPI_Allreduce` (où le trafic réseau est en $O(N^2)$, N étant la taille totale de la grille), la stratégie optimale consiste à utiliser une **décomposition spatiale**. Au lieu de diviser la population de fourmis, nous divisons **la carte elle-même** en sous-domaines, chaque processus MPI étant responsable d'une région spécifique (par un découpage en bandes 1D ou en blocs 2D).

Cette approche repose sur trois concepts architecturaux clés :

7.1. Échange de Halo (Ghost Cells)

Puisque le calcul de la diffusion des phéromones dépend des cellules voisines, chaque sous-domaine doit conserver en mémoire une bordure des régions adjacentes, désignée comme **cellules fantômes** (ghost cells). À chaque itération, les frontières des domaines sont synchronisées avec les voisins immédiats via des communications point-à-point (`MPI_Sendrecv` ou des requêtes asynchrones `MPI_Isend`/`MPI_Irecv`). **Avantage majeur :** Le volume de données échangées dépend uniquement du périmètre des sous-domaines, et non plus de la surface globale, abaissant radicalement la complexité des communications de $O(N^2)$ vers $O(N)$.

7.2. Migration des Fourmis

Les fourmis ne sont plus liées à un processus en permanence. Lorsqu'une fourmi franchit la lisière spatiale gérée par son rang actuel, elle effectue une **migration**. Cela implique la sérialisation de ses données (état, position relative, graine), sa transmission via MPI vers le nœud responsable du domaine d'arrivée, et sa suppression locale. C'est une communication directionnelle événementielle.

7.3. Équilibrage de Charge Évolutif (Load Balancing)

La nature biologique de l'implémentation engendre une densité spatiale très hétérogène (des paquets denses s'agglutinant sur les pistes entre le nid et la nourriture). Un algorithme aux frontières figées est exposé à de sérieux déséquilibres, où un unique nœud accaparent l'écrasante majorité de la charge CPU des déplacements de population. Pour contrer cette limite, il devient impératif d'intégrer un **équilibrage de charge dynamique**. Cela peut être implémenté via des redéploiements réguliers (en déplaçant les marges des domaines en fonction de la concentration de fourmis) ou en gérant de multiples plus petits lots spatiaux (*over-decomposition*) que l'on redistribue dynamiquement aux threads/noyaux libérés.

---

8. Conclusions

8.1 Conformité au Sujet

1. Comparaison par rapport au **code original** ;
2. Temps avec la **mémoire organisée (Step B)** en **1 noyau** ;
3. Memoire organisée + **OpenMP (Step C)** (multi-cœur).

Comparaison	Configuration	Total (s/itér)	Gain
Code original	OO, 1 thread	**1.426e-03**	référence
Step B	SoA, 1 thread	**1.483e-03**	**écart de 4.0%**
Step B + Step C	SoA, 8 threads	**7.513e-04**	**+47.3%**

8.2 Constat MPI

L'implémentation par Allreduce sur carte partagée provoque une limitation structurelle où les bénéfices CPU sont écrasés par la contrainte du bus mémoire du réseau.

8.3 Pistes de progression

La résolution efficace par MPI nécessite de découper la carte, en plus de la synchronisation de données aux marges spatiales.

---

9. Commandes de Lancement Rapide

Compiler

```
cd projet/src && make clean && make all
```

Étape C — OpenMP

```
OMP_PROC_BIND=true OMP_PLACES=cores OMP_DYNAMIC=false \
python3 projet/tools/bench.py --steps 1000 --runs 5 --threads 1 2 4 8 --
seed 2026
```

Étape D — MPI

```
mpirun -np 4 ./projet/src/ant_simu_mpi.exe --steps 300 --ants 20000 --seed
2026
```